DORA Reporting Platform

GitHub Repository Structure

Repo layout for dora-platform and dora-demo-app

Document ID: DORA-ARCH-0.2 | Date: 2026-03-13 | Status: Complete

Two-Repo Design

The project uses two GitHub repositories with distinct responsibilities:

Repo	Purpose	Visibility
dora-platform	Dashboard source + simulation scripts (this repo)	Public
dora-demo-app	Fake application repo populated by the simulator	Public

dora-platform — Directory Layout

```
dora-platform/
■
■■■ simulation/
■   ■■■ config.py              # API tokens, date ranges, volume params
■   ■■■ github_sim.py          # Creates commits/PRs/releases in dora-demo-app
■   ■■■ jira_sim.py            # Creates/transitions Jira tickets
■   ■■■ run_simulation.py      # Orchestrator — runs both scripts in order
■   ■■■ requirements.txt       # PyGithub, requests, python-dateutil
■
■■■ dashboard/
■   ■■■ index.html             # Single-page application entry point
■   ■■■ css/
■   ■   ■■■ style.css          # All dashboard styles
■   ■■■ js/
■       ■■■ config.js          # API base URLs (gitignored tokens)
■       ■■■ api.js             # fetch() wrappers for GitHub + Jira APIs
■       ■■■ metrics.js         # DORA metric computation logic
■       ■■■ charts.js          # Chart.js setup and rendering
■       ■■■ app.js             # Entry point — wires everything together
■
■■■ proxy/
■   ■■■ server.py              # Minimal proxy: handles Jira CORS + Basic Auth
■   ■■■ requirements.txt       # flask or just stdlib http.server
■
■■■ docs/                     # Architecture documents (this folder)
■   ■■■ 0.1_project_architecture.pdf
■   ■■■ 0.2_github_repo_structure.pdf
■   ■■■ 0.3_jira_project_setup.pdf
■   ■■■ 0.4_build_plan.pdf
■   ■■■ project_plan.md
■
■■■ .env.example              # Template: GITHUB_TOKEN, JIRA_TOKEN, etc.
■■■ .gitignore                # Ignores .env, __pycache__, *.pyc
```

```
■■■ README.md                  # Setup and run instructions
```

dora-demo-app — Directory Layout

This repo is populated entirely by github_sim.py. It simulates a real application codebase.

```
dora-demo-app/
■
■■■ src/
■   ■■■ main.py               # Fake application code (touched by commits)
■   ■■■ utils.py
■   ■■■ config.py
■
■■■ tests/
■   ■■■ test_main.py
■
■■■ CHANGELOG.md              # Updated on each simulated release
■■■ README.md
```

Key File Responsibilities

File	What it does
simulation/config.py	Central config: start/end dates, failure rate %, PR volume, Jira project key, repo names
simulation/github_sim.py	Uses PyGithub + local git to create backdated commits, open/merge PRs, create release tags
simulation/jira_sim.py	Uses Jira REST API to create stories/incidents, perform transitions with timestamps
simulation/run_simulation.py	Runs github_sim then jira_sim, links Jira tickets to GitHub PRs via branch name
dashboard/js/api.js	All fetch() calls — encapsulates GitHub + Jira endpoints, pagination, error handling
dashboard/js/metrics.js	Pure functions: takes raw API responses, returns aggregated DORA metric arrays by week/mon
dashboard/js/charts.js	Chart.js configuration, DORA band overlay lines, weekly/monthly dataset swap
proxy/server.py	Listens on localhost:8080, forwards /jira/* to Jira Cloud with Basic Auth header injected

Branching Convention in dora-demo-app

The simulator creates branches following a naming pattern so Jira tickets can be linked:

Branch pattern	Example	Linked Jira ticket
feature/DORA-{n}-{slug}	feature/DORA-42-add-login	DORA-42 (Story)
hotfix/DORA-{n}-{slug}	hotfix/DORA-87-fix-crash	DORA-87 (Incident)
release/v{x.y.z}	release/v1.4.0	Tagged as GitHub release